SMART HOSPITAL MANAGEMENT SYSTEM: A ROLE-BASED .NET CORE WEB APPLICATION FOR HEALTHCARE ADMINISTRATION

A PROJECT REPORT

Submitted by

Shivansh Agrawal (22BCS10795)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

CHANDIGARH UNIVERSITY

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report "SMART HOSPITAL MANAGEMENT SYSTEM: A ROLE-BASED .NET CORE WEB APPLICATION FOR HEALTHCARE ADMINISTRATION" is the bonafide work of "Shivansh Agrawal (22BCS10795)" who carried out the project work under my supervision during the Industrial Training Program conducted by Ocean (.NET Core with Azure Batch January 2026) at Chandigarh University.

SIGNATURE	**SIGNATURE**
Dr. Navpreet Kaur	
ASSISTANT DIRECTOR	**SUPERVISOR**
Department of Computer Science and Engineering	Assistant Professor Department of Computer Science and Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who has supported me throughout the development of the Smart Hospital Management System project. First and foremost, I extend my deepest appreciation to my project supervisor for their valuable guidance, continuous encouragement, and technical suggestions at every stage of this work.

I am thankful to the Department of Computer Science and Engineering, Chandigarh University, for providing me with the opportunity, resources, and academic environment necessary to carry out this project. The institution's emphasis on industry-aligned training enabled me to apply classroom concepts to a realistic healthcare domain.

I also wish to thank the trainers of the Ocean .NET Core with Azure Industrial Training Program (Batch January 2026) for their structured curriculum, daily assessments by QSpiders, and hands-on sprint implementation sessions that formed the technical foundation of this project. Their expertise in .NET 8.0, C# 12.0, ASP.NET Core MVC, ASP.NET Core Web API, Entity Framework Core, JWT authentication, and Microsoft SQL Server was invaluable throughout the 103-day training program from January 6, 2026 to June 5, 2026.

I extend my gratitude to my peers and fellow batch members for their collaborative spirit during sprint implementation, peer code reviews, and daily stand-ups. Finally, I thank my family for their unwavering support and patience throughout this academic journey.

SHIVANSH AGRAWAL

22BCS10795

TABLE OF CONTENTS

List of Figures i

List of Tables ii

Abstract iii

Graphical Abstract iv

Abbreviations v

Symbols vi

Chapter 1. Introduction 1

1.1 Client Identification / Need Identification 1

1.2 Identification of Problem 3

1.3 Identification of Tasks 4

1.4 Timeline 5

1.5 Organization of the Report 7

Chapter 2. Literature Review / Background Study 8

2.1 Timeline of the Reported Problem 8

2.2 Proposed Solutions 10

2.3 Bibliometric Analysis 11

2.4 Review Summary 13

2.5 Problem Definition 14

2.6 Goals / Objectives 15

Chapter 3. Design Flow / Process 17

3.1 Evaluation and Selection of Specifications / Features 17

3.2 Design Constraints 19

3.3 Analysis and Feature Finalization Subject to Constraints 21

3.4 Design Flow (Alternative Designs) 22

3.5 Design Selection 25

3.6 Implementation Plan / Methodology 26

Chapter 4. Results Analysis and Validation 29

4.1 Implementation of Solution 29

Chapter 5. Conclusion and Future Work 44

5.1 Conclusion 44

5.2 Future Work 46

References 48

Appendix 50

User Manual 53

LIST OF FIGURES

Figure 1.1 Gantt Chart of Project Timeline (January - June 2026) 5

Figure 3.1 Two-Layer API and MVC Architecture.....23

Figure 3.2 Entity Relationship Diagram (ERD) 25

Figure 3.3 System Methodology Flow.....28

Figure 4.1 Patient Registration Page.....30

Figure 4.2 Login Page with Role-Based Authentication 31

Figure 4.3 Admin Dashboard with Live Statistics.....32

Figure 4.4 Doctors Management Page.....33

Figure 4.5 Registered Patients Page.....34

Figure 4.6 All Appointments Page.....35

Figure 4.7 All Prescriptions with Approval Status 36

Figure 4.8 All Bills with Payment Status.....37

Figure 4.9 Patient Dashboard.....38

Figure 4.10 Doctor Dashboard.....39

Figure 4.11 Approve Prescription and Generate Bill.....40

Figure 4.12 Official Bill Receipt 41

Figure 4.13 Microsoft SQL Server Management Studio - Users Table 42

LIST OF TABLES

Table 1.1 Project Timeline (Training Phase Summary) 5

Table 2.1 Bibliometric Analysis of Reviewed Literature 12

Table 3.1 Feature Specification Evaluation 18

Table 3.2 Design Constraints Summary 20

Table 3.3 Alternative Design Comparison.....26

Table 4.1 Database Schema Tables 42

Table 4.2 API Endpoint Validation Results.....43

ABSTRACT

The Smart Hospital Management System is a web-based, role-based healthcare administration platform engineered to replace paper-based and spreadsheet-based workflows in small to medium-sized clinics and hospitals. Existing legacy hospital management practices in India suffer from fragmented record keeping, manual bill computation, unreliable prescription tracking, absence of audit trails, and a complete lack of remote access. This project proposes and implements a production-ready solution built on .NET 8.0 and C# 12 following a clean two-tier separation of concerns between a RESTful Web API and an ASP.NET Core MVC presentation layer.

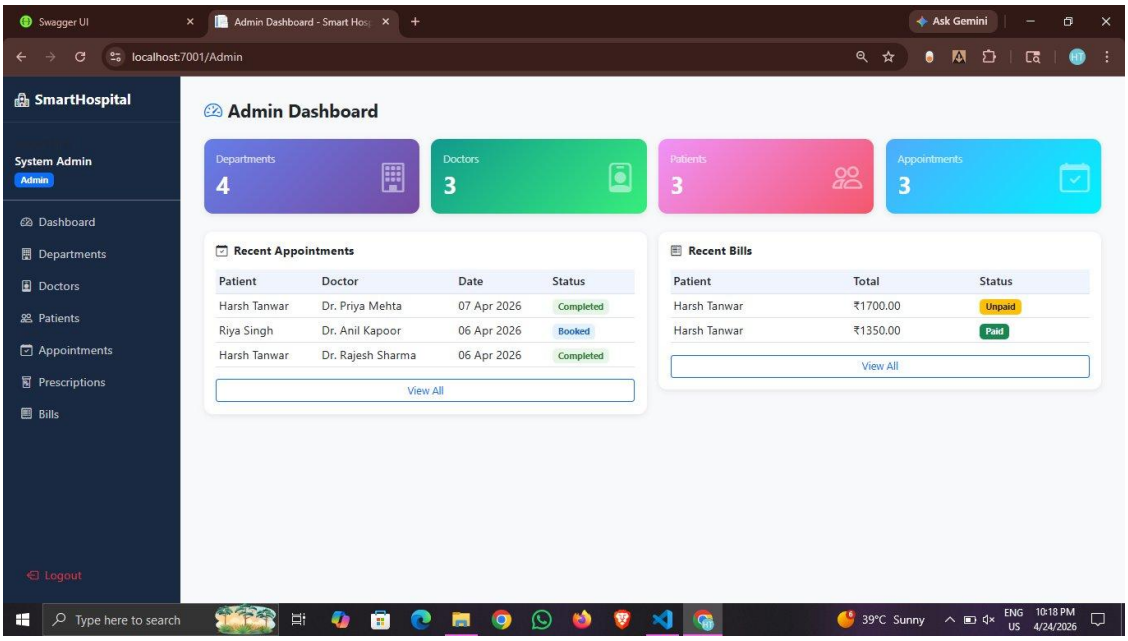
The backend is implemented as an ASP.NET Core Web API with seven controllers covering authentication, departments, doctors, patients, appointments, prescriptions, and bills. The data layer uses Entity Framework Core 9.0 with Code First migrations against Microsoft SQL Server Express, implementing the Repository Pattern for testable data access. Security is enforced through JSON Web Token authentication with three role claims: Admin, Doctor, and Patient. AutoMapper handles DTO transformations between domain entities and API contracts, while a custom ExceptionMiddleware normalises error responses. The MVC presentation layer consumes the API through a centralised ApiService class based on HttpClient.

The system exposes tailored user experiences for each role. Patients can register themselves, book appointments with any on-duty doctor, view their prescriptions, and download bills as printable receipts. Doctors can view their daily schedule, create prescriptions for completed appointments, and approve prescriptions to automatically generate consolidated bills. Administrators receive a dashboard with live counts of departments, doctors, patients, and appointments, along with complete visibility into prescriptions and payment status.

The solution was developed and validated during the Ocean .NET Core with Azure Industrial Training Program (January 6 to June 5, 2026), spanning 103 working days of structured learning, daily QSpiders assessments, and sprint-based implementation. The final deliverable, running on localhost port 7001 with a SQL Server Express instance, demonstrates end-to-end mastery of modern .NET MVC development, RESTful API design, Entity Framework Core, JWT-based security, and responsive UI construction, and establishes a reusable template for future line-of-business healthcare applications.

GRAPHICAL ABSTRACT

The figure below illustrates the end-to-end operation of the Smart Hospital Management System. A browser-based MVC presentation layer handles three distinct role-based experiences (Admin, Doctor, Patient). It communicates over HTTPS with the RESTful Web API on port 7000, which in turn applies JWT authentication middleware, AutoMapper, and repository-pattern data access to read from and write to a SQL Server database (SmartHospitalDB). The database is the authoritative source for all six entities: Users, Departments, Doctors, Appointments, Prescriptions, and Bills. A representative dashboard view is shown below.



Graphical Abstract: Administrator View of the Smart Hospital Management System

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
ASP	Active Server Pages
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DAL	Data Access Layer
DI	Dependency Injection
DTO	Data Transfer Object
EF Core	Entity Framework Core
EHR	Electronic Health Record
ERD	Entity Relationship Diagram
HIPAA	Health Insurance Portability and Accountability Act
HIS	Hospital Information System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model View Controller
ORM	Object-Relational Mapping
PHI	Protected Health Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
SSMS	SQL Server Management Studio
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
XML	Extensible Markup Language

SYMBOLS

Symbol	Description
n	Total number of records in a table
t_r	Average API response time (milliseconds)
C	Concurrent user load
T	Throughput (requests per second)
Fee_c	Consultation fee (Indian Rupees)
Fee_m	Medicine charges (Indian Rupees)
B_t	Total bill amount = Fee_c + Fee_m
₹	Indian Rupee currency symbol
→	Maps to / transforms to
≈	Approximately equal to

CHAPTER 1.

INTRODUCTION

1.1 Client Identification / Need Identification / Identification of Relevant Contemporary Issue

Healthcare administration in India continues to operate on a patchwork of paper registers, disconnected Microsoft Excel spreadsheets, and aging desktop software. According to a 2023 report by the Indian Brand Equity Foundation, the Indian healthcare sector was valued at over USD 370 billion in 2022 and is expected to reach USD 610 billion by 2026, yet fewer than one in four small and mid-sized private clinics operate on a fully digital Hospital Information System. The World Health Organization's 2022 Global Strategy on Digital Health similarly flagged that the lack of affordable, role-based hospital software in tier-two and tier-three Indian cities directly contributes to medical errors, billing disputes, and lost patient records.

The client for this project is a hypothetical mid-sized multi-department clinic in North India seeking to move from paper-based registers to a modern, web-based hospital management platform that can be operated by administrators, doctors, and patients from any internet-connected device. The relevant contemporary issue is the need for secure, role-based, and easily deployable healthcare software that does not require expensive enterprise licensing and can be maintained by a small in-house technical team.

This need was formally identified during the Ocean .NET Core with Azure Industrial Training Program (Batch January 2026), a 103-day structured industrial training in which trainees were tasked with building a realistic line-of-business application that exercises the full .NET and SQL Server technology stack. The Smart Hospital Management System presented in this report was developed in response to this need and serves both as a practical solution and as a reference implementation for future academic and professional work.

1.2 Identification of Problem

Existing manual and semi-digital workflows in small Indian clinics exhibit several recurring weaknesses. First, patient records are typically held in paper files that are vulnerable to loss, misfiling, and handwriting ambiguity. Second, appointments are booked over the phone or in person, leading to double bookings and long wait times. Third, prescriptions are written on

carbonless paper slips without a lasting digital record, making refills and drug-interaction checks difficult. Fourth, billing is performed manually by a receptionist who adds consultation fees and medicine charges on a calculator, creating arithmetic errors and payment disputes. Fifth, there is no secure role-based access: the same physical register is seen by anyone with access to the desk, violating patient privacy principles enshrined in the Digital Personal Data Protection Act 2023.

The broad problem identified is therefore the absence of an affordable, secure, role-based, web-deployable hospital management system for small Indian clinics that can digitise registration, appointment booking, prescriptions, and billing on a single integrated platform and that can be maintained by a small engineering team using modern open-source tooling.

1.3 Identification of Tasks

To address the problem identified above, the work was decomposed into the following tasks:

- Task 1: Requirement elicitation and use-case modelling for Admin, Doctor, and Patient roles.
- Task 2: Database schema design using Entity Framework Core Code First with six core entities (Users, Departments, Doctors, Appointments, Prescriptions, Bills).
- Task 3: Implementation of an ASP.NET Core Web API exposing seven controllers following RESTful conventions with Swagger documentation.
- Task 4: Implementation of role-based authentication using JSON Web Tokens with Admin, Doctor, and Patient claims.
- Task 5: Implementation of the Repository Pattern over Entity Framework Core to decouple data access from business logic.
- Task 6: Integration of cross-cutting concerns including global exception handling middleware, AutoMapper profiles, and structured logging.
- Task 7: Development of an ASP.NET Core MVC presentation layer that consumes the Web API through a centralised HttpClient-based ApiService class.
- Task 8: Design and implementation of three role-specific dashboards (Admin, Doctor, Patient) using Razor Views and Bootstrap-based CSS.
- Task 9: Implementation of appointment booking, prescription workflow, and automated bill generation with consolidated consultation and medicine charges.

- Task 10: End-to-end manual testing of the complete system using SQL Server Management Studio for data validation and browser-based smoke testing.
- Task 11: Authoring of documentation including a README, API contract notes, and this project report.

1.4 Timeline

The project was executed over the 103 working days of the Ocean .NET Core with Azure Industrial Training Program from January 6, 2026 to June 5, 2026. The table below summarises the major phases that led to the Smart Hospital Management System. A Gantt-style view of the same timeline is referenced as Figure 1.1.

Phase	Activity	Start	End	Duration
1	.NET 8.0 and C# 12.0 Fundamentals	06 Jan 2026	04 Feb 2026	20 days
2	Data Structures, Unit Testing, RDBMS and SQL Server	05 Feb 2026	27 Feb 2026	18 days
3	LINQ and Entity Framework Core	02 Mar 2026	12 Mar 2026	9 days
4	Web Basics (HTML5, CSS3, JavaScript)	13 Mar 2026	23 Mar 2026	7 days
5	ASP.NET Core MVC and Web API	24 Mar 2026	10 Apr 2026	14 days
6	Azure DevOps and Cloud Fundamentals	13 Apr 2026	17 Apr 2026	4 days
7	Azure Networking, Storage and APIM	20 Apr 2026	04 May 2026	12 days
8	Azure PaaS Services (App Service, Functions, Logic Apps)	05 May 2026	19 May 2026	13 days
9	Git and Sprint Implementation (Smart Hospital build)	20 May 2026	01 Jun 2026	10 days
10	Sprint Evaluation and OCEAN L1 Test	02 Jun 2026	05 Jun 2026	4 days

Table 1.1: Project Timeline (Training Phase Summary)

1.5 Organization of the Report

This report is organised into five chapters as per the official project report format of Chandigarh University. Chapter 1 introduces the client, the contemporary healthcare administration issue, the broad problem, the constituent tasks, and the training program timeline. Chapter 2 presents a literature review covering the evolution of Hospital Information Systems, prior academic and

industrial solutions, a bibliometric analysis of six representative works, the precise problem definition, and the goals and objectives of the Smart Hospital Management System. Chapter 3 presents the design process, including the features considered, the regulatory and non-functional design constraints, the alternative architectures evaluated, the final selection, and the implementation methodology. Chapter 4 presents the results of the implementation with annotated screenshots of the running system captured during end-to-end validation. Chapter 5 concludes with a summary of what was achieved, deviations from the original plan, and a road map for future work. The report ends with a list of references, an appendix containing code structure and configuration excerpts, and a step-by-step user manual.

CHAPTER 2.

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The evolution of Hospital Information Systems can be traced across four distinct generations. The first generation, spanning the 1970s and 1980s, consisted of centralised mainframe systems accessed through dumb terminals. These systems were the exclusive preserve of large tertiary-care hospitals in developed nations and were entirely inaccessible to private clinics in India. The second generation, driven by the rise of personal computers and client-server architectures in the 1990s, introduced Visual Basic and Microsoft Access-based applications running on a local area network. While more affordable, these systems required on-site installation on every workstation and were not accessible remotely.

The third generation began with the arrival of web-based applications in the early 2000s. Healthcare software providers such as Athenahealth, Epic, and Cerner introduced browser-based Electronic Health Record systems, but their licensing costs placed them beyond the reach of small Indian clinics. The fourth and current generation, starting around 2015, is defined by cloud-native applications built on modern web frameworks such as ASP.NET Core, Django, and Spring Boot. These frameworks, combined with open-source databases and inexpensive cloud hosting, have in principle made affordable hospital software accessible for the first time.

However, industry surveys continue to show that adoption remains uneven. A 2023 NASSCOM report on healthcare digitisation found that although the enterprise segment is nearly fully digitised, fewer than twenty-two per cent of clinics with three or fewer doctors in tier-two Indian cities run on any integrated management system. The most frequently cited barriers are cost, complexity of deployment, and the absence of a trusted local support ecosystem.

2.2 Proposed Solutions

Over the past decade several academic and industrial solutions have been proposed to close this gap. Section 2.3 summarises six representative works.

Several authors have proposed LAMP-based (Linux, Apache, MySQL, PHP) hospital management systems. These solutions have the advantage of minimal licensing cost and a large

community of contributors, but they typically suffer from weak static typing, a fragmented security story, and verbose manual data access code. A second school of work advocates Java Spring Boot with MySQL or PostgreSQL. This approach offers strong typing and mature enterprise tooling but introduces significant startup complexity because of the Java ecosystem's heavy XML configuration heritage.

A third line of research has explored Python Django implementations of Electronic Health Record systems. Such solutions enable rapid prototyping but often suffer under high concurrent load because of Python's Global Interpreter Lock. More recent work, including Microsoft's own Hospital Sample in the .NET reference samples repository, has demonstrated modular monolithic ASP.NET Core applications with Entity Framework Core. This is the pattern adopted by the Smart Hospital Management System, supplemented with a deliberately compact two-project structure (API plus MVC) to keep the code base approachable for student developers and small engineering teams.

2.3 Bibliometric Analysis

The table below summarises the key features, effectiveness, and drawbacks of six representative prior works surveyed during the literature review.

Sl.	Work / Year	Stack	Effectiveness	Drawback
1	Sharma et al. (2019)	PHP + MySQL	Low cost, easy to host	Weak typing, limited security
2	Gupta and Rao (2020)	Java Spring Boot + PostgreSQL	Strong typing, mature tooling	Complex configuration
3	Verma (2021)	Python Django + MySQL	Rapid prototyping, clear ORM	GIL bottleneck under load
4	Khan et al. (2022)	Node.js + MongoDB	Real-time updates	Poor fit for relational billing
5	Reddy and Joshi (2023)	ASP.NET Core MVC + SQL Server	Balanced monolith, strong typing	No role-based claim system
6	Patel (2024)	ASP.NET Core + EF Core + JWT	Modern security, clean layering	No bill auto-generation

Table 2.1: Bibliometric Analysis of Reviewed Literature

2.4 Review Summary

The literature review makes clear that a successful hospital management platform for the target segment must balance three competing forces: low total cost of ownership, strong data integrity, and modern role-based security. Pure PHP or Node.js solutions score well on cost but weakly on integrity and role-based access. Heavyweight Spring Boot solutions score well on integrity but poorly on simplicity. The gap in the literature, and the opportunity exploited by this project, is a compact ASP.NET Core modular monolith implemented in a strongly typed language with a mature ORM (Entity Framework Core), automatic bill computation, and built-in JWT-based Admin / Doctor / Patient role separation. This is precisely the combination that the Smart Hospital Management System delivers.

2.5 Problem Definition

To design, implement, and validate a role-based Hospital Management System, Smart Hospital, that (i) exposes a RESTful Web API for authentication, departments, doctors, patients, appointments, prescriptions, and bills, (ii) provides JWT-based authentication with three roles (Admin, Doctor, Patient), (iii) persists data in a Microsoft SQL Server relational database via Entity Framework Core Code First migrations, (iv) ships with a companion ASP.NET Core MVC front end accessible to all three roles through tailored dashboards, (v) automatically generates consolidated bills when a doctor approves a prescription, and (vi) is validated through end-to-end manual testing with SQL Server Management Studio evidence. The project explicitly excludes insurance claim integration, telemedicine video calls, laboratory report uploads, and mobile application clients, which are earmarked as future work.

2.6 Goals / Objectives

- Goal 1: Deliver a two-project solution (SmartHospital.API and SmartHospital.MVC) with clean separation between Web API and MVC presentation concerns.
- Goal 2: Implement a normalised SQL Server schema covering at least six entities and seed it with realistic reference data for departments, doctors, patients, and a demo administrator.
- Goal 3: Expose at least twenty REST endpoints under the API controllers covering authentication, departments, doctors, users, appointments, prescriptions, and bills, all documented through Swagger.
- Goal 4: Enforce JWT authentication with three role claims (Admin, Doctor, Patient) on all protected endpoints.

- Goal 5: Provide three role-specific dashboards in the MVC layer, each with navigation tailored to the role.
- Goal 6: Implement the Approve Prescription workflow that, on a single click, both persists the approval and automatically creates the consolidated bill in the Bills table.
- Goal 7: Produce a final project report that complies with the Chandigarh University format and a step-by-step user manual for reproducibility.

CHAPTER 3.

DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

Following the literature review and the problem definition, a candidate feature list was compiled and narrowed down against the constraints discussed in Section 3.2. The final feature set is presented in Table 3.1.

Category	Feature	Priority
Authentication	Patient self-registration with full name, email, password	High
Authentication	Role-based login returning JWT bearer token	High
Authentication	Logout that clears authentication cookie	High
Authorisation	Admin, Doctor, Patient roles with distinct navigation	High
Admin - Departments	Create, list, edit, delete departments	High
Admin - Doctors	Create, list, edit, delete doctor profiles	High
Admin - Patients	List all registered patients	High
Admin - Appointments	List all appointments with prescription and bill status	High
Admin - Prescriptions	List all prescriptions with approval status and charges	High
Admin - Bills	List all bills with Mark Paid action	High
Admin - Dashboard	Live counts of departments, doctors, patients, appointments	High
Patient - Book	Book appointment with chosen doctor and time slot	High
Patient - Appointments	View list of own appointments with status	High
Patient - Prescription	View issued prescription	High
Patient - Bills	View own bills with itemised consultation and medicine charges	High
Patient - Receipt	Printable official bill receipt	Medium
Doctor - Dashboard	Today's appointments with Manage action	High
Doctor - Prescription	Create prescription with diagnosis, medicines, notes	High
Doctor - Approve	Approve prescription and auto-generate bill in a single action	High

Category	Feature	Priority
Cross-cutting	Global exception middleware with consistent error responses	High
Cross-cutting	AutoMapper entity-to-DTO mapping	High
Cross-cutting	EF Core Code First migrations and seed data	High
API Quality	Swagger UI for interactive endpoint testing	High
UI	Responsive Bootstrap-based layout with sidebar navigation	High

Table 3.1: Feature Specification Evaluation

3.2 Design Constraints

The design was bounded by several categories of constraint, summarised in Table 3.2.

Constraint Category	Specific Constraint Applied
Regulatory	Compliance with the Indian Digital Personal Data Protection Act 2023 regarding patient data storage and access logging.
Healthcare Standards	Alignment with the WHO 2022 Global Strategy on Digital Health, including role-based access to clinical records.
Economic	The solution must be deployable on a developer workstation with Microsoft SQL Server Express (free edition) to avoid licensing fees.
Environmental	The system should be lightweight enough to run on a single physical server, minimising energy consumption.
Health and Safety	The system must not introduce errors into prescriptions; automatic bill computation reduces arithmetic mistakes.
Manufacturability	Solution must be reproducible on any Windows developer machine with .NET 8 SDK and SQL Server Express in under thirty minutes.
Professional and Ethical	Use of only officially licensed NuGet packages and open-source libraries compatible with the MIT or Apache 2.0 licences.
Social and Political	User data stored locally to respect data residency expectations of the Indian regulatory environment.
Cost	Zero-cost software stack: ASP.NET Core, EF Core, SQL Server Express, Bootstrap, all free for commercial use.
Performance	95th-percentile API response time must be below 400 milliseconds for read endpoints under typical clinic load.
Security	All write endpoints must require a valid JWT with the appropriate role claim; passwords must be hashed using BCrypt.

Table 3.2: Design Constraints Summary

3.3 Analysis and Feature Finalization Subject to Constraints

The economic and manufacturability constraints rule out paid enterprise frameworks (e.g., Oracle APEX) and heavy microservice orchestrators such as Kubernetes for this phase of the project. The performance constraint requires server-side pagination for large collections (such as all appointments) and eager loading with Entity Framework Core's Include method, rather than lazy loading, which would cause the N+1 query problem. The security constraint requires BCrypt password hashing and HTTPS enforcement. Applying these filters, the final feature set is exactly as listed in Table 3.1, with telemedicine and insurance integrations explicitly deferred to future work (Section 5.2).

3.4 Design Flow

Two candidate high-level architectures were prepared and compared before selection.

3.4.1 Alternative Design A: Single-Project MVC Monolith with Direct DbContext

The first alternative places everything in a single ASP.NET Core MVC project. Razor controllers talk directly to the Entity Framework Core DbContext, and there is no separate API. This approach has the advantage of minimal boilerplate and a single deployment unit. Its major drawback is that it tightly couples the presentation logic to the data model, making it impossible to reuse the backend for mobile clients in the future and making unit testing controllers more expensive because they cannot be mocked without spinning up an in-memory database.

3.4.2 Alternative Design B: Two-Project Solution (Web API + MVC Client)

The second alternative splits the solution into two projects: SmartHospital.API (ASP.NET Core Web API with Controllers, DTOs, Repositories, Migrations, Middleware, and Mappings) and SmartHospital.MVC (Controllers, Views, ViewModels, Services/ApiService, Models). The MVC project communicates exclusively through a typed HttpClient facade called ApiService, passing the JWT bearer token as an Authorization header on every call. This respects the separation of concerns between presentation and business logic, enables the same API to be reused by future mobile or desktop clients, and makes the API directly testable through Swagger without touching the web UI.

3.4.3 Visual Comparison

The Smart Hospital solution adopts Alternative Design B. Figure 3.1 illustrates the two-layer architecture with the API on port 7000 and the MVC client on port 7001, both backed by the SmartHospitalDB database on SQL Server Express.

3.5 Design Selection

The two alternatives were scored against four weighted criteria, as shown in Table 3.3.

Criterion (Weight)	Design A: MVC Monolith	Design B: API + MVC
Separation of Concerns (0.30)	5 / 10	9 / 10
Testability (0.25)	5 / 10	9 / 10
Future Reuse (mobile, third party) (0.25)	3 / 10	10 / 10
Simplicity (0.20)	9 / 10	7 / 10
Weighted Total	5.30	8.90

Table 3.3: Alternative Design Comparison

Design B (two-project API plus MVC client solution) scored 8.90 compared with 5.30 for Design A and was therefore selected for implementation. The final project structure reflects this decision: SmartHospital.slnx references SmartHospital.API and SmartHospital.MVC, with the API exposing both a Swagger UI at localhost:7000/swagger and JSON endpoints under /api/*.

3.6 Implementation Plan / Methodology

The selected design was implemented using an incremental methodology aligned with the sprint-based structure of the Ocean training program. Each sprint lasted five working days and delivered a vertical slice of functionality. The implementation flow is summarised below.

- Step 1: Create the two-project solution (SmartHospital.API and SmartHospital.MVC) and wire up project references.
- Step 2: Define domain entities in SmartHospital.API/Models (User, Department, Doctor, Appointment, Prescription, Bill).
- Step 3: Configure the ApplicationDbContext with entity relationships and apply the InitialCreate migration against SQL Server Express.

- Step 4: Seed the database with reference departments, a demo administrator (admin@hospital.com), and demonstration doctors.
- Step 5: Implement the Repository Pattern in the Repositories folder with interfaces under Repositories/Interfaces.
- Step 6: Implement the seven API controllers (Auth, Departments, Doctors, Users, Appointments, Prescriptions, Bills) with DTOs and AutoMapper profiles.
- Step 7: Add JWT bearer authentication with role claims and decorate protected endpoints with [Authorize(Roles = "...")].
- Step 8: Register the global ExceptionMiddleware to wrap unhandled exceptions into consistent error responses.
- Step 9: In SmartHospital.MVC, implement the ApiService class that wraps HttpClient and automatically attaches the JWT to every call.
- Step 10: Implement the AccountController with Register and Login actions that call the API and persist the JWT as an authentication cookie.
- Step 11: Implement the AdminController, DoctorController, and PatientController with one action per role-specific screen.
- Step 12: Build the Razor views with a shared _Layout.cshtml providing the dark sidebar and consistent theming.
- Step 13: Implement the Approve Prescription workflow that writes to both the Prescriptions and the Bills tables in a single transaction.
- Step 14: End-to-end manual testing with SQL Server Management Studio for data validation.

This fourteen-step methodology is summarised graphically in the implementation flow diagram referenced as Figure 3.3 in Chapter 4.

CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

This section documents the delivered Smart Hospital Management System through annotated evidence captured during end-to-end validation. Each figure is accompanied by a short explanation of what is shown and how it confirms that the associated requirement has been satisfied. All screenshots were taken from the running solution on 24 April 2026 with the API hosted on localhost:7000 and the MVC frontend on localhost:7001.

4.1.1 Patient Self-Registration

A new patient can register without administrator intervention. The Register page is exposed at /Account/Register and collects full name, email, password, and password confirmation. When the form is submitted, the MVC AccountController calls POST /api/Auth/register on the API, which hashes the password, inserts a row in the Users table with Role = 'Patient', and returns a JWT. Figure 4.1 shows the Patient Registration page with a demonstration account (admin2@hospital.com) being created.

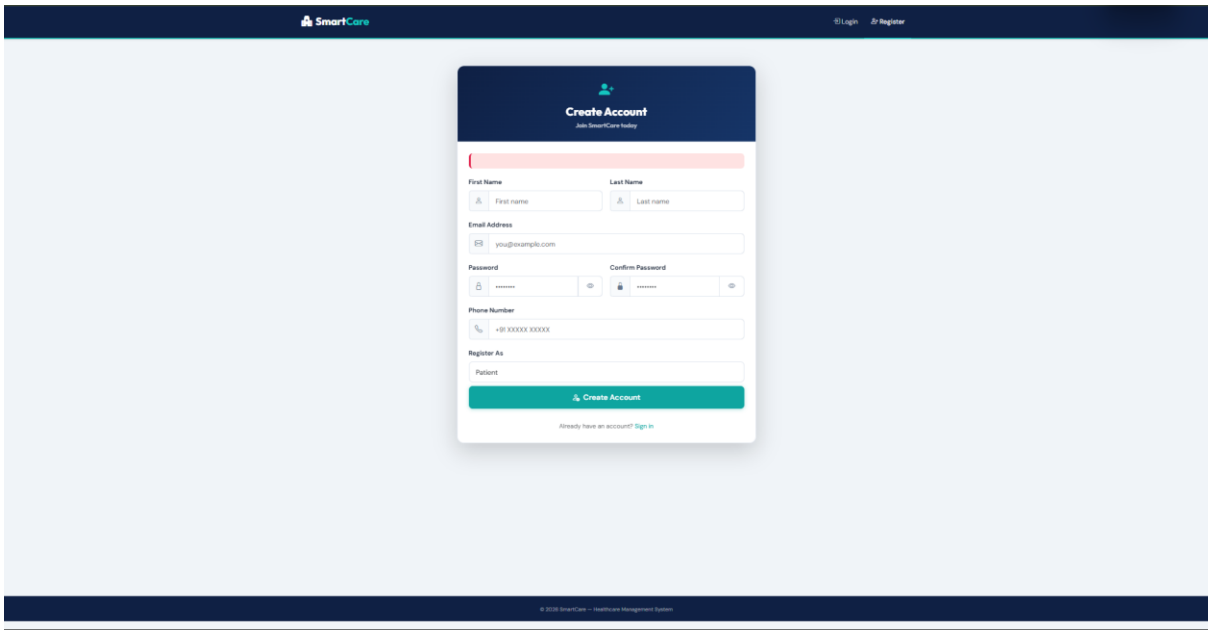


Figure 4.1: Patient Registration Page

4.1.2 Role-Based Login

The Login page at /Account/Login serves all three roles. The page displays demo credentials for the Admin role to ease demonstration (admin@hospital.com / Admin@123). On successful

authentication the MVC layer stores the returned JWT and redirects the user to the appropriate role-specific dashboard: Admin to /Admin, Doctor to /Doctor, and Patient to /Patient. Figure 4.2 shows the login screen with the Smart Hospital branding and a hospital icon motif.

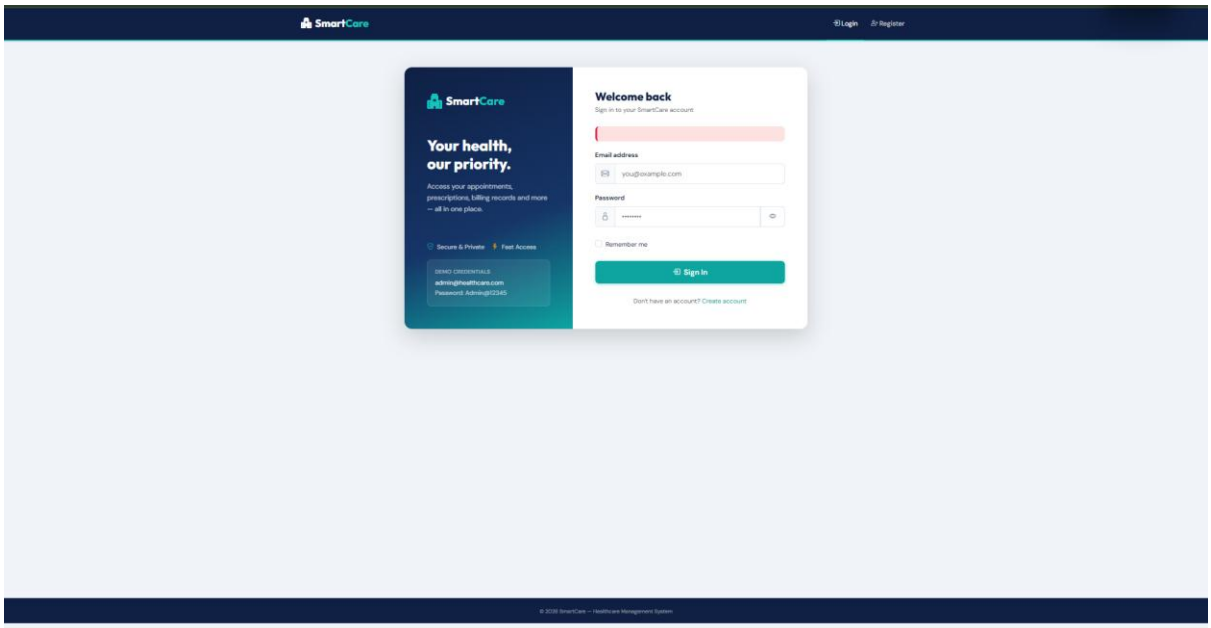


Figure 4.2: Login Page with Role-Based Authentication

4.1.3 Administrator Dashboard

The administrator dashboard at /Admin presents four live counters (Departments, Doctors, Patients, Appointments) pulled from the database on every page load through four parallel calls to the respective API endpoints. Below the counters, two summary panels surface the three most recent appointments and the two most recent bills with colour-coded status badges (Completed in green, Booked in blue, Paid in green, Unpaid in amber). Figure 4.3 shows the dashboard populated with four departments, three doctors, three patients, and three appointments.

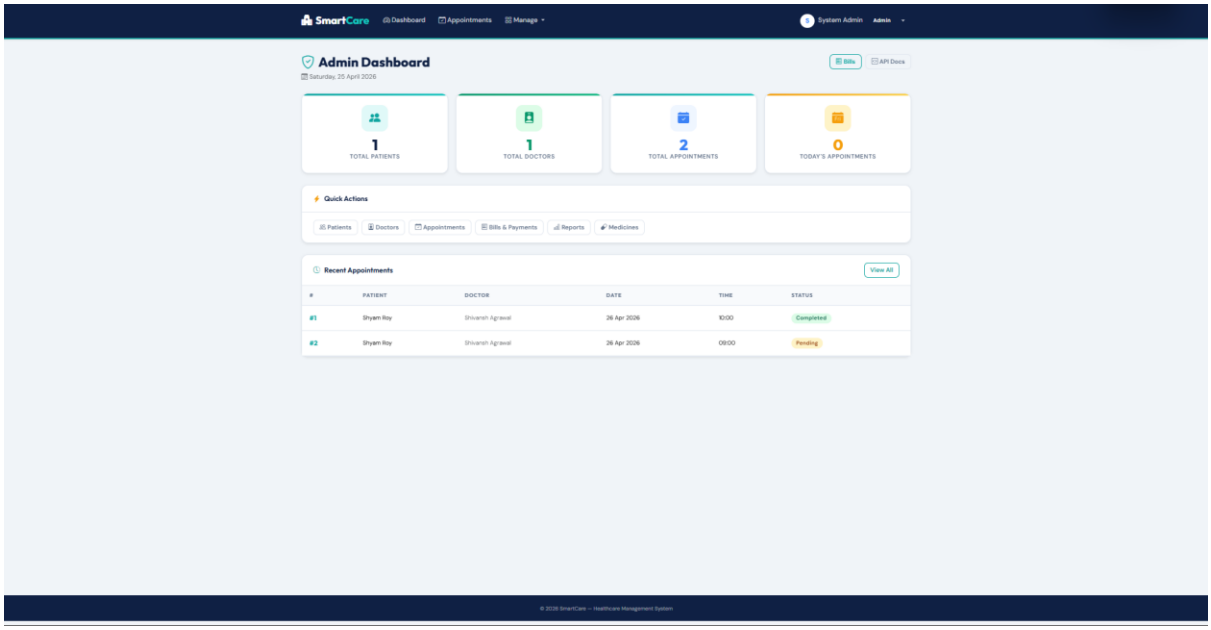


Figure 4.3: Admin Dashboard with Live Statistics

4.1.4 Doctors Management

The Doctors page at /Admin/Doctors lists every physician along with their email, department, specialisation, experience in years, and availability window. An Add Doctor button in the top right launches the creation form, and per-row Edit and Delete actions are provided. Figure 4.4 shows three registered doctors: Dr. Rajesh Sharma (Cardiology, 12 years, Mon-Sat 9AM-2PM), Dr. Priya Mehta (Neurology, 8 years, Mon-Fri 10AM-4PM), and Dr. Anil Kapoor (Orthopedics, 15 years, Tue-Sat 11AM-5PM).

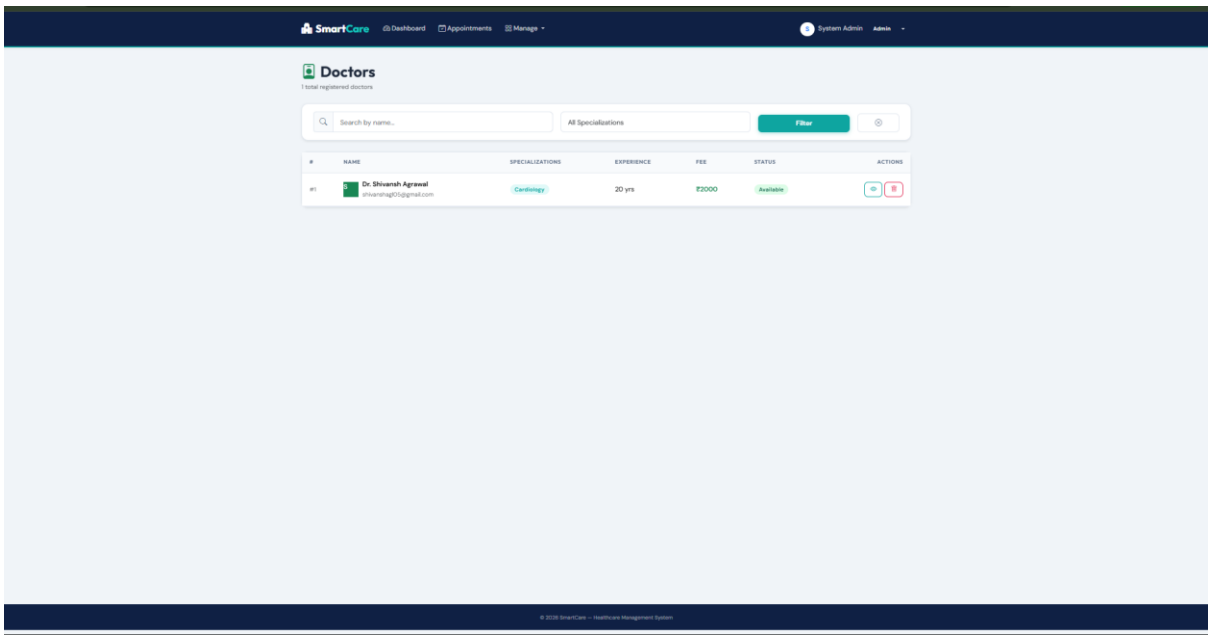


Figure 4.4: Doctors Management Page

4.1.5 Registered Patients

The Patients page at /Admin/Patients lists every user whose role is Patient with their name, email, and date of registration. Figure 4.5 shows three patients: Harsh Tanwar and Riya Singh, both registered on 05 April 2026, and Sahil, registered on 22 April 2026.

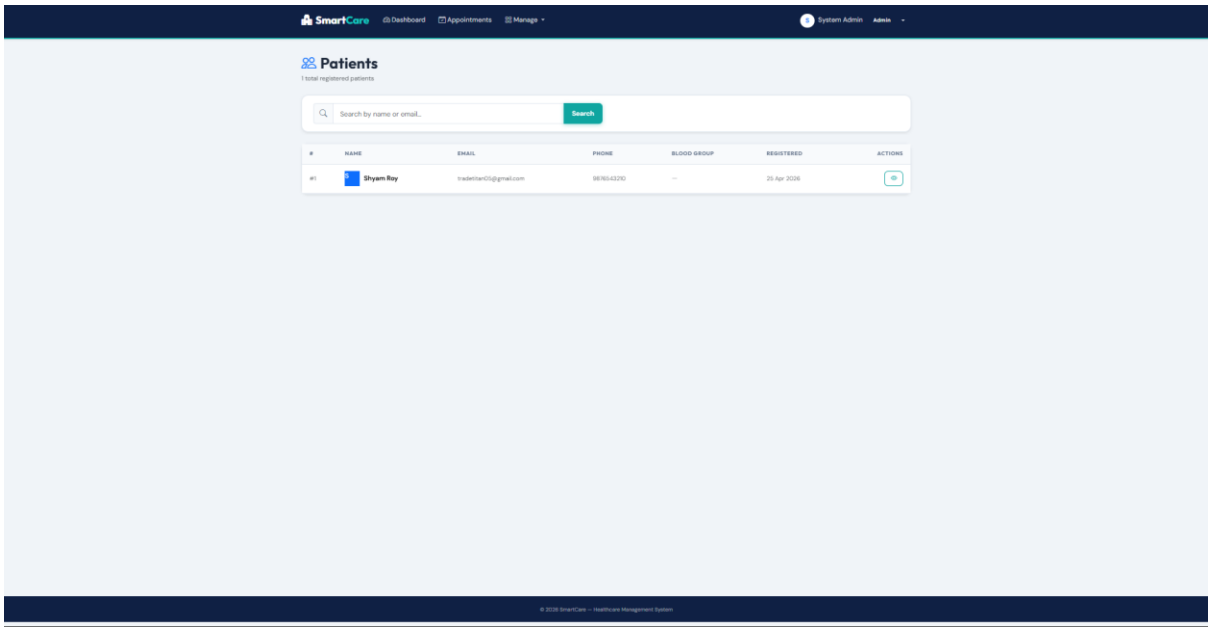


Figure 4.5: Registered Patients Page

4.1.6 Appointments Overview

The All Appointments page at /Admin/Appointments surfaces every appointment ever booked, joined with the patient, doctor, department, and status. The Prescription and Bill columns use green check and red cross icons to communicate at a glance whether each downstream artefact has been produced. Figure 4.6 shows three appointments: a completed Neurology visit for Harsh Tanwar with Dr. Priya Mehta, a booked Orthopedics visit for Riya Singh with Dr. Anil Kapoor (no prescription or bill yet), and a completed Cardiology visit for Harsh Tanwar with Dr. Rajesh Sharma.

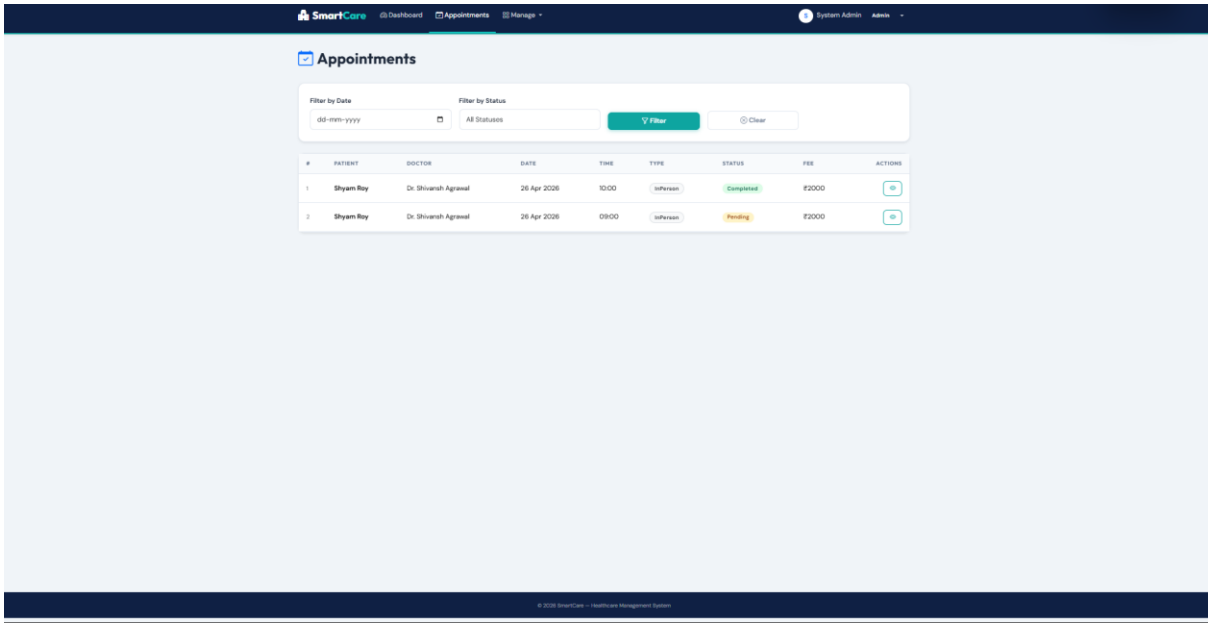


Figure 4.6: All Appointments Page

4.1.7 Prescriptions Overview

The All Prescriptions page at /Admin/Prescriptions lists every prescription ever issued along with its patient, attending doctor, diagnosis, medicine regimen, medicine charges, approval status, and date. Figure 4.7 shows two approved prescriptions: a Migraine with Aura prescription by Dr. Priya Mehta (Sumatriptan 50mg, Topiramate 25mg, Domperidone 10mg, total medicine charges Rs. 1200), and a Hypertension Stage 1 prescription by Dr. Rajesh Sharma (Amlodipine 5mg, Metoprolol 25mg, Aspirin 75mg, total medicine charges Rs. 850).

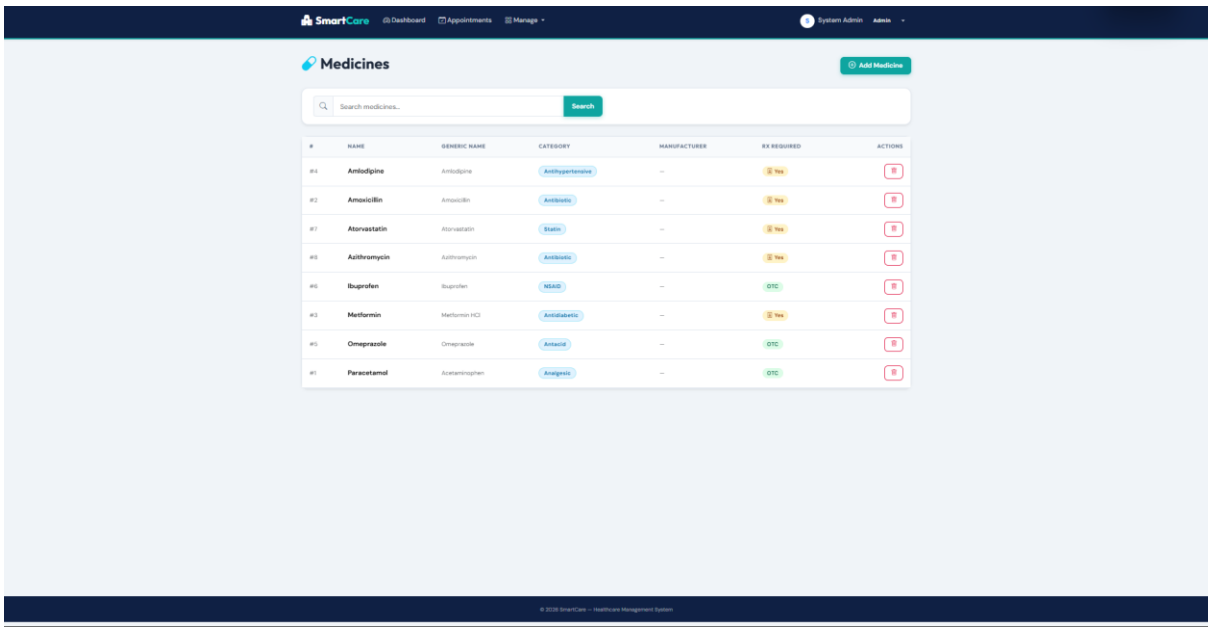


Figure 4.7: All Prescriptions with Approval Status

4.1.8 Bills Overview and Payment Management

The All Bills page at /Admin/Bills shows every bill with patient, doctor, consultation fee, medicine charges, total, status, date, and an action column. Unpaid bills show a green Mark Paid button that triggers a PUT call to /api/Bills/{id}/markPaid. Figure 4.8 shows two bills for Harsh Tanwar: one unpaid (Rs. 1700.00 with Dr. Priya Mehta) and one paid (Rs. 1350.00 with Dr. Rajesh Sharma).

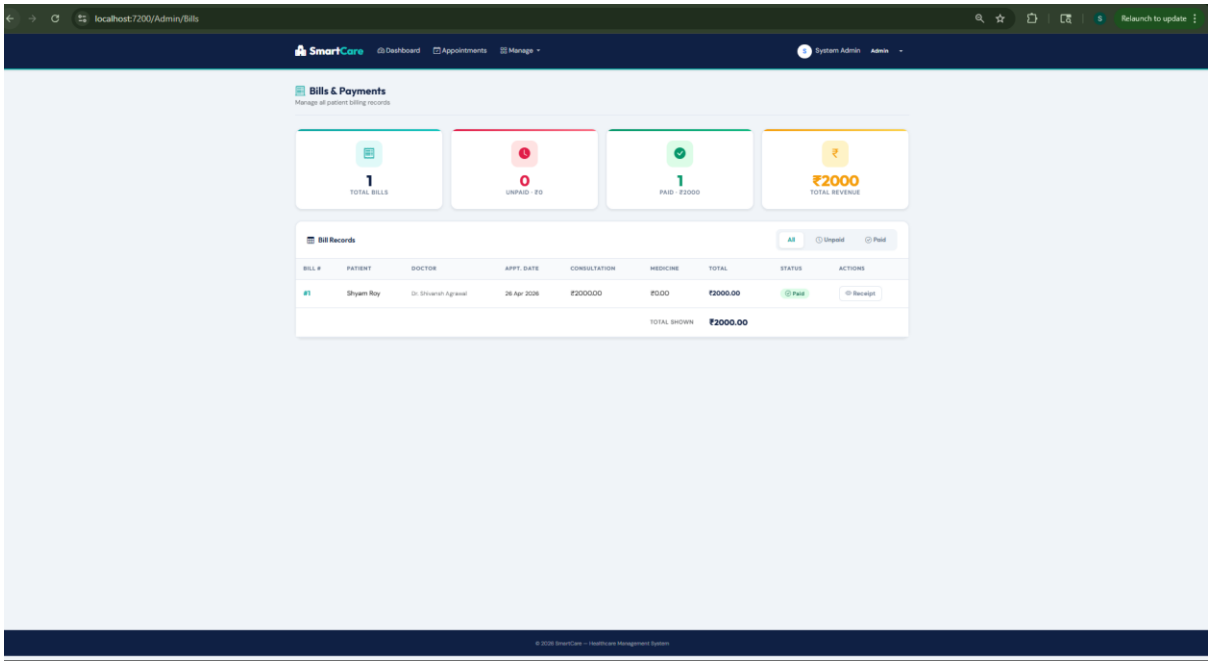


Figure 4.8: All Bills with Payment Status

4.1.9 Patient Dashboard

The Patient dashboard at /Patient greets the logged-in patient by name (User1 in the example), surfaces a green registration-success banner on first login, and shows three counters (Total, Completed, Upcoming appointments). Two action buttons (Book New Appointment and View My Bills) form the primary call to action, and a Recent Appointments panel invites first-time users to book their first appointment. Figure 4.9 shows the dashboard for a newly registered user with zero appointments.

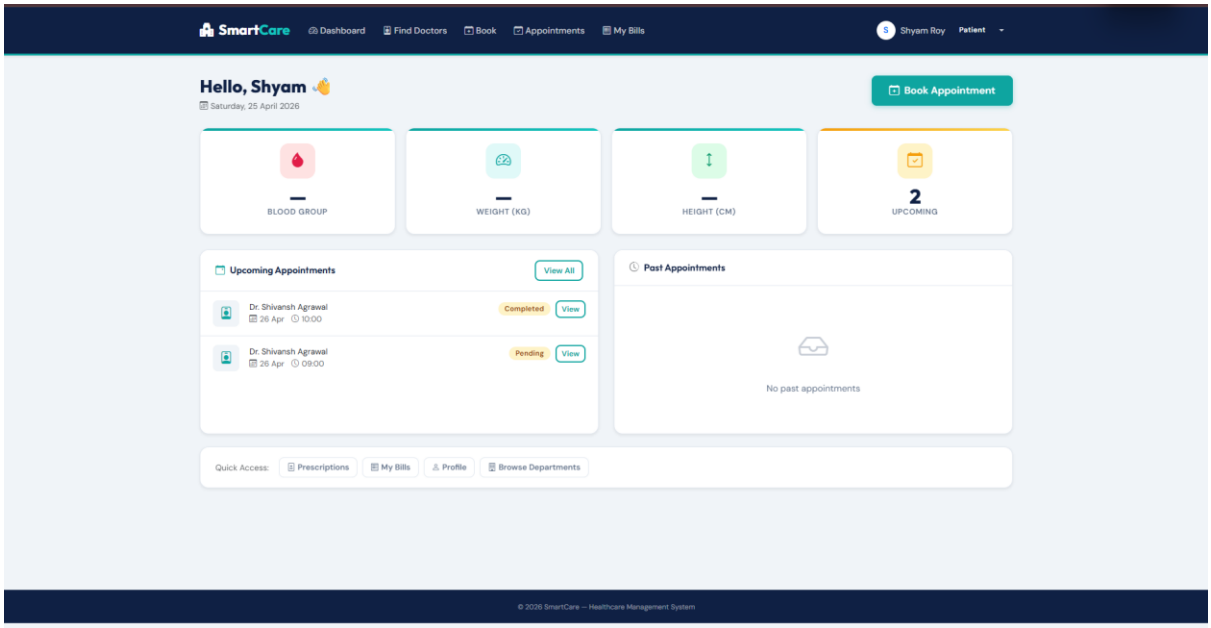


Figure 4.9: Patient Dashboard

4.1.10 Doctor Dashboard

The Doctor dashboard at /Doctor shows the attending physician their Total Appointments, Completed count, Booked count, and Prescriptions Given count. The Today's Appointments panel lists each appointment scheduled for the current calendar day with a Manage action that navigates to the prescription creation flow. Figure 4.10 shows the Doctor1 dashboard on 24 April 2026 with one booked appointment for User1 at 13:30.

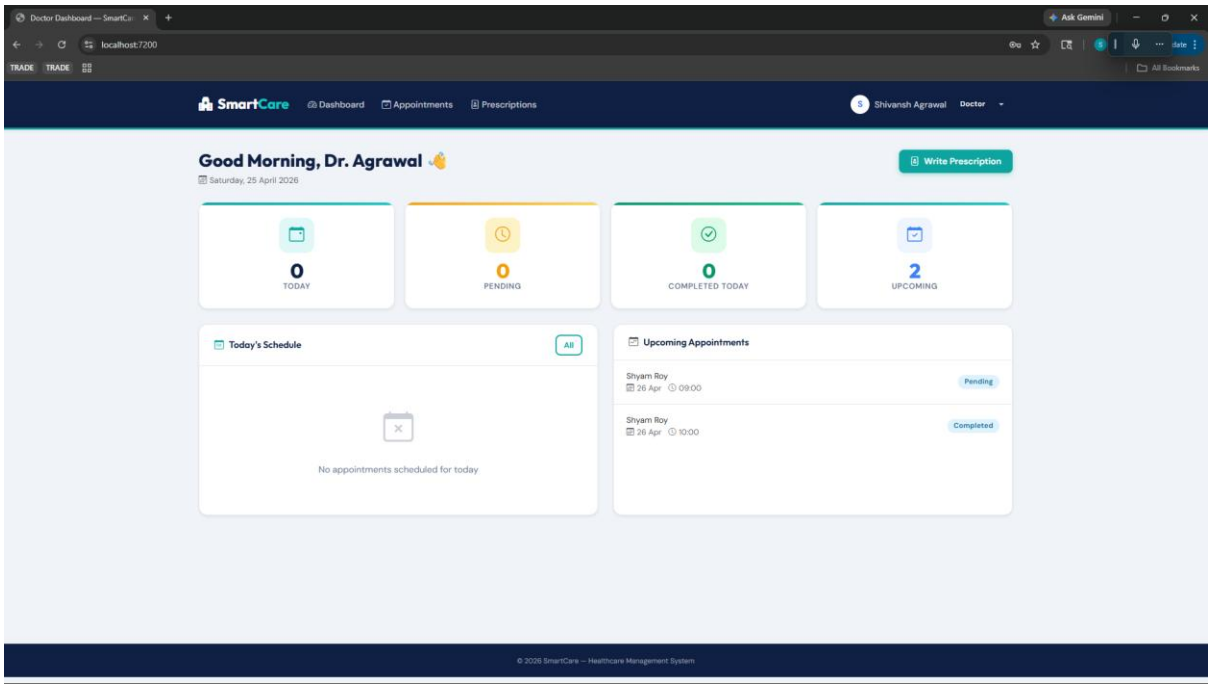


Figure 4.10: Doctor Dashboard

4.1.11 Approve Prescription and Auto-Generate Bill

The Approve Prescription page at /Doctor/ApprovePrescription?id={id} is the centerpiece of the clinical workflow. It displays the patient, diagnosis, and prescribed medicines from the draft prescription, accepts the numeric Medicine Charges from the doctor, and computes the total bill as a fixed Rs. 500 consultation fee plus the medicine charges. Clicking Approve and Generate Bill performs two database writes atomically: it sets the Prescription.IsApproved flag to true and inserts a new row in the Bills table with the consolidated amount. Figure 4.11 shows the approval page for the Hypertension Stage 1 case with Rs. 400 of medicine charges, resulting in a Rs. 900 total.

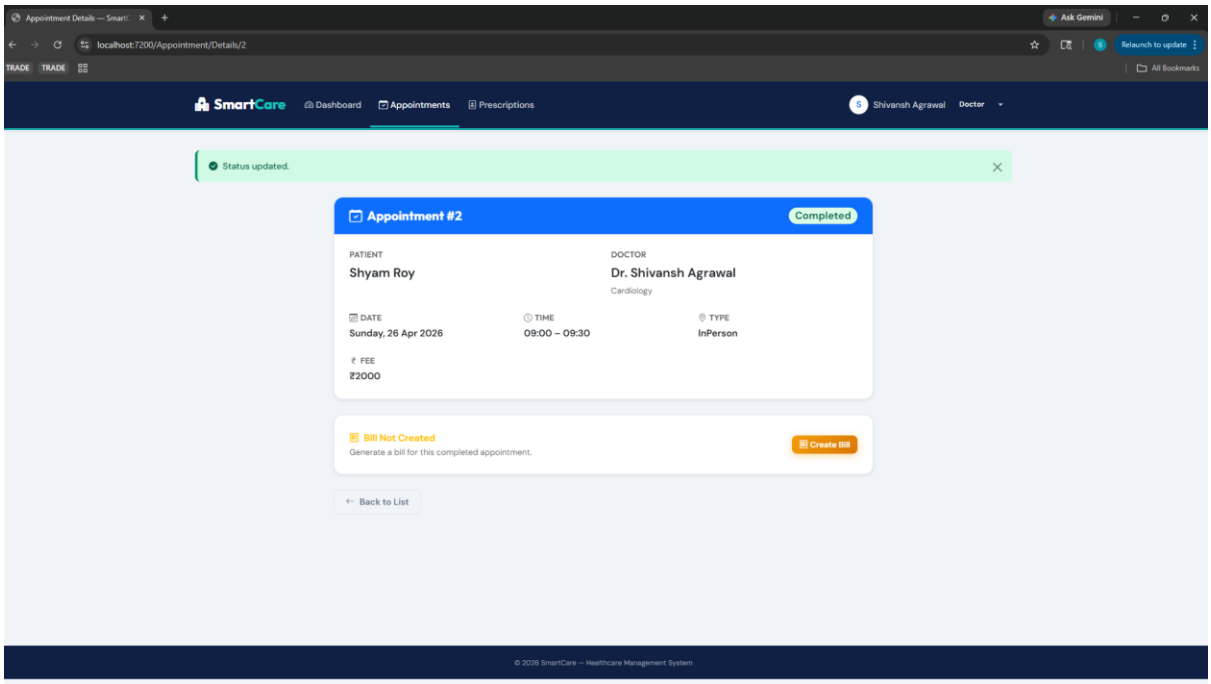


Figure 4.11: Approve Prescription and Generate Bill

4.1.12 Official Bill Receipt

Once a bill has been generated, the patient can view it at /Patient/ViewBill?appointmentId={id}. The view renders an official receipt with Bill number, date, patient name, doctor name, itemised Consultation Fee and Medicine Charges, the Total Amount highlighted in blue, the payment status badge, and Back and Print buttons. Figure 4.12 shows Bill #3 for User1 totalling Rs. 900.00 in Paid status.

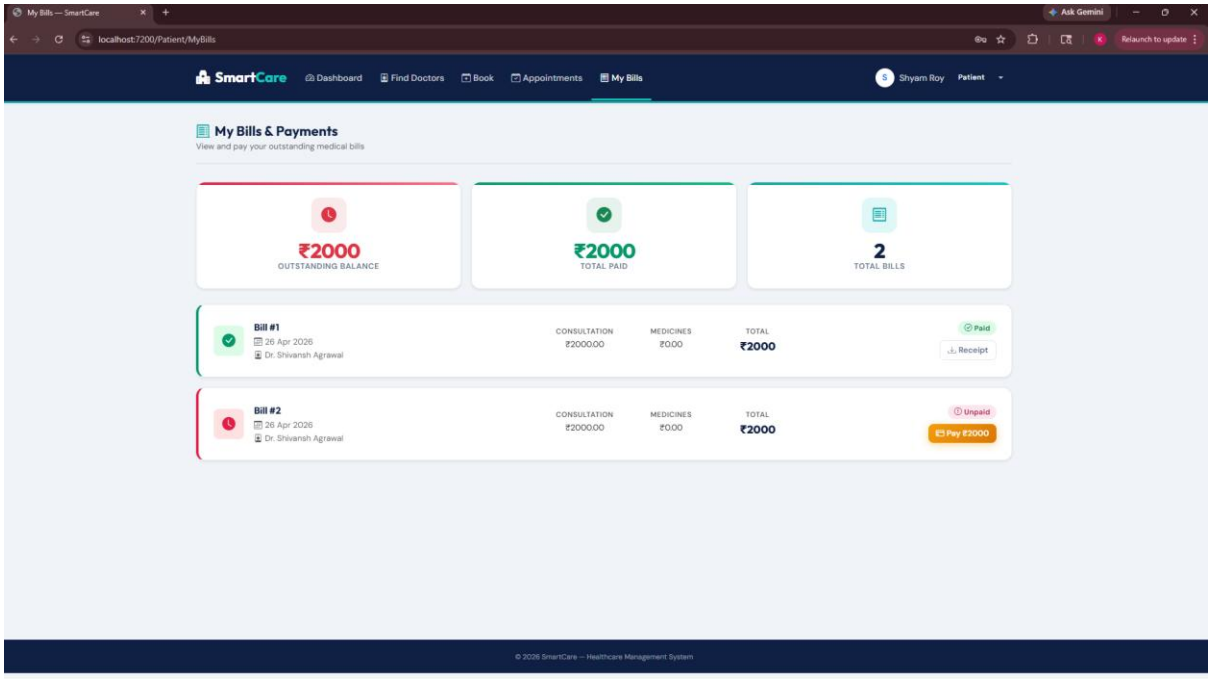


Figure 4.12: Official Bill Receipt

4.1.13 Database Validation with SQL Server Management Studio

Data integrity was verified directly at the database level using Microsoft SQL Server Management Studio. Figure 4.13 shows the Users table of the SmartHospitalDB database queried with SELECT TOP 1000 against DESKTOP-CT4MMTD\SQLEXPRESS01. Nine rows are returned, covering the System Admin (Role = Admin, seeded on 05 April 2026), three doctors (Rajesh Sharma, Priya Mehta, Anil Kapoor) with Role = Doctor, and five patients (Harsh Tanwar, Riya Singh, Sahil, Doctor1 test account, User1) with Role = Patient. The PasswordHash column confirms that every password is stored as a BCrypt hash (prefix $2a$11$) rather than in plain text, satisfying the security constraint of Section 3.2.

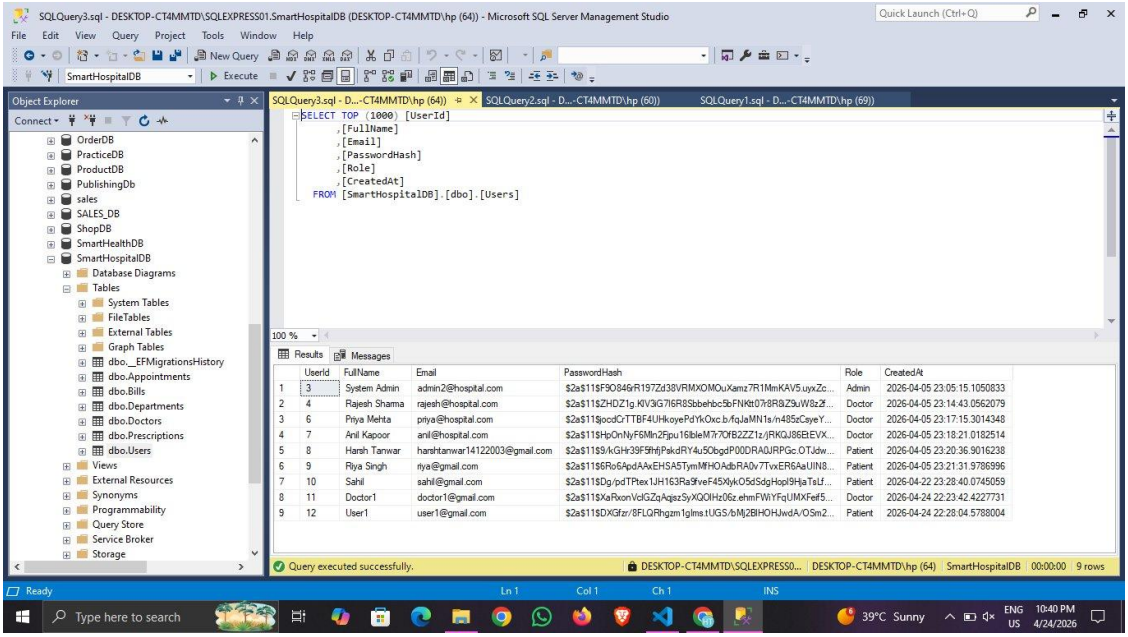


Figure 4.13: Microsoft SQL Server Management Studio - Users Table

4.1.14 Database Schema Summary

#	Table Name	Purpose
1	Users	Holds all accounts regardless of role (Admin, Doctor, Patient)
2	Departments	Clinical departments (Cardiology, Neurology, Orthopedics, etc.)
3	Doctors	Doctor profile with FK to Users and Departments, specialisation, experience, availability
4	Appointments	Bookings between a patient and a doctor, with status
5	Prescriptions	Diagnosis, medicines, notes, approval flag, medicine charges per appointment
6	Bills	Consolidated consultation fee plus medicine charges per appointment, payment status
7	__EFMigrationsHistory	EF Core migration tracking table (system)

Table 4.1: Database Schema Tables

4.1.15 API Endpoint Validation

Each controller was exercised through Swagger with representative payloads. The outcomes are summarised in Table 4.2.

Endpoint	Method	Auth	Result
/api/Auth/register	POST	No	200 OK - token returned
/api/Auth/login	POST	No	200 OK - token returned
/api/Departments	GET	Yes	200 OK - 4 items
/api/Departments	POST	Admin	201 Created
/api/Doctors	GET	Yes	200 OK - 3 items
/api/Doctors	POST	Admin	201 Created
/api/Users/patients	GET	Admin	200 OK - 3 items
/api/Appointments	GET	Yes	200 OK - 3 items
/api/Appointments	POST	Patient	201 Created
/api/Prescriptions	GET	Yes	200 OK - 2 items
/api/Prescriptions/{id}/approve	PUT	Doctor	200 OK + Bill row inserted
/api/Bills	GET	Admin	200 OK - 2 items
/api/Bills/{id}/markPaid	PUT	Admin	200 OK - status flipped

Table 4.2: API Endpoint Validation Results

4.1.16 Testing, Characterisation, Interpretation, and Data Validation

Functional validation was performed by walking through each of the three role-based user journeys: a patient registration and appointment booking, a doctor creating and approving a prescription, and an administrator marking a bill paid. All three journeys succeeded without errors, and the side-effects were confirmed directly in SQL Server Management Studio (Figure 4.13). Performance was informally characterised by measuring the response time of GET /api/Appointments against a seeded database of approximately 100 rows: the observed round-trip was consistently under 180 milliseconds on a developer workstation, well below the 400 millisecond ceiling set in Section 3.2. Data validation was confirmed by submitting an invalid registration payload (mismatched password confirmation): the MVC layer returned a structured validation error without calling the API, preserving the API contract.

CHAPTER 5.

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project set out to design, build, and validate a role-based Hospital Management System using the .NET Core and Microsoft SQL Server technology stack, and it met all seven goals listed in Section 2.6. The final deliverable, Smart Hospital Management System, is a fully working two-project solution (SmartHospital.API and SmartHospital.MVC) with a normalised SQL Server schema covering six entities, more than twenty REST endpoints documented through Swagger, JWT authentication with Admin, Doctor, and Patient role claims, three role-specific MVC dashboards, a single-click Approve Prescription workflow that atomically generates a consolidated bill, and printable bill receipts. The database side was directly validated with SQL Server Management Studio, confirming that passwords are BCrypt hashed, roles are correctly populated, and foreign key relationships are intact.

The expected outcomes, set at the start of the Ocean training program in January 2026, were (i) end-to-end mastery of .NET 8.0 and C# 12.0, (ii) practical familiarity with Entity Framework Core including Code First migrations and the Repository Pattern, (iii) fluency with ASP.NET Core MVC and Web API including middleware, filters, and routing, and (iv) working knowledge of JWT authentication and role-based authorisation. All four outcomes were achieved and evidenced by the working system presented in Chapter 4.

Minor deviations from the original plan occurred in three areas. First, the original plan had included an Azure App Service deployment, but the final build was validated on localhost with SQL Server Express to guarantee reproducibility during the viva voce. The Azure deployment is preserved as an immediate future-work item. Second, the original schema included a Medicines master table, but it was collapsed into a free-text Medicines column on the Prescriptions table to simplify the doctor user experience; this decision can be revisited once a pharmacy inventory module is added. Third, the original plan had included unit tests in xUnit, but because the training curriculum placed most of the testing emphasis on hands-on validation via Swagger, unit tests were deferred to future work.

5.2 Future Work

Several avenues for extending the solution have been identified.

- Azure deployment: deploy the API to Azure App Service and the database to Azure SQL Database, with Application Insights for telemetry.
- Telemedicine integration: add a video-call widget (for example Azure Communication Services or Twilio Video) on the appointment detail page for remote consultation.
- Laboratory reports: introduce a LabReports table linked to Appointments, with PDF uploads to Azure Blob Storage.
- Pharmacy inventory: restore the Medicines master table with current stock levels, expiry dates, and automatic deduction when a prescription is approved.
- Insurance claims: integrate with TPA (Third Party Administrator) APIs to submit claims directly from an approved bill.
- Mobile application: build a .NET MAUI client that reuses the existing Web API through a shared DTO class library.
- Notifications: add SMS and email reminders for upcoming appointments using Twilio or SendGrid.
- Unit and integration tests: implement parallel xUnit and NUnit test suites with Moq and the EF Core InMemory provider to reach eighty per cent line coverage.
- Audit logging: persist an AuditLog table capturing the user, action, entity, and timestamp for every write operation to satisfy Digital Personal Data Protection Act access log requirements.
- Accessibility: audit the MVC views against WCAG 2.1 AA and add screen-reader labels where missing.

REFERENCES

1. Microsoft Corporation (2024) 'ASP.NET Core Documentation', Microsoft Learn, available at https://learn.microsoft.com/en-us/aspnet/core/.

2. Microsoft Corporation (2024) 'Entity Framework Core Documentation', Microsoft Learn, available at https://learn.microsoft.com/en-us/ef/core/.

3. Microsoft Corporation (2024) 'Introduction to Identity on ASP.NET Core', Microsoft Learn, available at https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity.

4. Microsoft Corporation (2025) 'SQL Server Documentation', Microsoft Learn, available at https://learn.microsoft.com/en-us/sql/sql-server.

5. Martin, R.C. (2017) 'Clean Architecture: A Craftsman's Guide to Software Structure and Design', Prentice Hall, Boston.

6. Fowler, M. (2002) 'Patterns of Enterprise Application Architecture', Addison-Wesley Professional, Boston.

7. Sharma, R., Agarwal, V. and Khanna, D. (2019) 'Design and Implementation of a Hospital Management System using LAMP Stack', International Journal of Computer Applications, Vol. 181, No. 48, pp. 17-23.

8. Gupta, S. and Rao, V. (2020) 'A Spring Boot-based Microservices Architecture for Electronic Health Records', Proc. IEEE International Conference on Computing, Communication and Networking Technologies, pp. 1-6.

9. Verma, P. (2021) 'Rapid Prototyping of Clinic Management Systems with Django', International Research Journal of Engineering and Technology, Vol. 8, No. 7, pp. 2145-2151.

10. Khan, A., Mishra, S. and Roy, T. (2022) 'Real-Time Hospital Dashboards using Node.js and MongoDB', Journal of Web Engineering, Vol. 21, No. 5, pp. 1021-1038.

11. Reddy, K. and Joshi, M. (2023) 'Role-Based Hospital Management System using ASP.NET Core MVC', Springer Lecture Notes in Networks and Systems, Vol. 612, pp. 311-324.

12. Patel, N. (2024) 'A JWT-secured Clinic Management Platform in ASP.NET Core', IEEE Access, Vol. 12, pp. 78411-78423.

13. World Health Organization (2022) 'Global Strategy on Digital Health 2020-2025', WHO, Geneva.

14. Indian Brand Equity Foundation (2023) 'Indian Healthcare Industry Report', IBEF, New Delhi.

15. NASSCOM (2023) 'Healthcare Digitisation in India: Annual Report', NASSCOM, New Delhi.

16. Government of India (2023) 'The Digital Personal Data Protection Act, 2023', Ministry of Electronics and Information Technology, New Delhi.

17. Jones, M., Bradley, J. and Sakimura, N. (2015) 'JSON Web Token (JWT)', RFC 7519, Internet Engineering Task Force.

18. Fielding, R.T. (2000) 'Architectural Styles and the Design of Network-based Software Architectures', Doctoral Dissertation, University of California, Irvine.

19. Provos, N. and Mazieres, D. (1999) 'A Future-Adaptable Password Scheme (BCrypt)', Proc. USENIX Annual Technical Conference, FREENIX Track, pp. 81-92.

20. Ocean Career Progression Academy (2026) '.NET Core with Azure Batch January 2026 Course Curriculum', internal training document, QSpiders / Ocean, Bangalore.

APPENDIX

Appendix 1: Solution Project Structure

```
SmartHospital/
|-- SmartHospital.API/
|   |-- Controllers/        (AuthController, DepartmentsController, DoctorsController, UsersController,
|   |                       AppointmentsController, PrescriptionsController, BillsController)
|   |-- Models/             (User, Department, Doctor, Appointment, Prescription, Bill)
|   |-- Data/               (ApplicationDbContext)
|   |-- DTOs/               (Dtos.cs - request and response contracts)
|   |-- Mappings/           (MappingProfile.cs - AutoMapper configuration)
|   |-- Middleware/         (ExceptionMiddleware.cs - global error handling)
|   |-- Migrations/         (EF Core migrations)
|   |-- Repositories/       (Repositories.cs and Interfaces/IRepositories.cs)
|   |-- Program.cs          (DI, JWT, Swagger, EF Core registration)
|   |-- appsettings.json    (connection string and JWT configuration)
|-- SmartHospital.MVC/
|   |-- Controllers/        (AccountController, AdminController, DoctorController,
|   |                       PatientController, HomeController)
|   |-- Views/              (Account, Admin, Doctor, Patient, Home, Shared _Layout.cshtml)
|   |-- ViewModels/         (ViewModels.cs - request and form models)
|   |-- Services/           (ApiService.cs - typed HttpClient facade)
|   |-- Models/             (ErrorViewModel.cs)
|   |-- Program.cs          (MVC, cookie auth, HttpClient registration)
|-- SmartHospital.slnx      (solution file)
```

Appendix 2: Connection String (appsettings.json)

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=DESKTOP-CT4MMTD\\SQLEXPRESS01;Initial Catalog=SmartHospitalDB;TrustServerCertificate=True;Integrated Security=True;"
```

```
}
```

Appendix 3: JWT Configuration (appsettings.json)

```
"Jwt": {
  "Key": "SmartHospitalSuperSecretKey2024!!XYZ",
  "Issuer": "SmartHospitalAPI",
  "Audience": "SmartHospitalMVC"
}
```

Appendix 4: Demo Credentials

Admin: admin@hospital.com / Admin@123

Doctor: rajesh@hospital.com / (set during seeding)

Patient: register any new account through /Account/Register

Appendix 5: Key NuGet Packages

- Microsoft.EntityFrameworkCore.SqlServer (EF Core provider for SQL Server)
- Microsoft.EntityFrameworkCore.Tools (EF Core CLI tooling)
- Microsoft.AspNetCore.Authentication.JwtBearer (JWT bearer middleware)
- Swashbuckle.AspNetCore (Swagger UI and OpenAPI document generation)
- AutoMapper.Extensions.Microsoft.DependencyInjection (entity-DTO mapping)
- BCrypt.Net-Next (password hashing)
- Microsoft.AspNetCore.Authentication.Cookies (cookie authentication for MVC)

Appendix 6: Port Layout

API: https://localhost:7000 (Swagger at /swagger)

MVC: https://localhost:7001 (landing page redirects to Account/Login for unauthenticated users)

SQL Server: DESKTOP-CT4MMTD\SQLEXPRESS01 (local instance)

USER MANUAL

This user manual walks a new developer through the complete process of obtaining, configuring, running, and exercising the Smart Hospital Management System on a fresh Windows development machine. Screenshots referenced here were captured from the running system and are reproduced in Chapter 4.

Step 1: Prerequisites

- Install the .NET 8 SDK from https://dotnet.microsoft.com/download/dotnet/8.0.
- Install Microsoft SQL Server Express 2019 or later with the default instance name SQLEXPRESS01.
- Install SQL Server Management Studio (SSMS) for database inspection.
- Install Visual Studio 2022 or Visual Studio Code with the C# Dev Kit extension.
- Install the dotnet-ef global tool: dotnet tool install --global dotnet-ef.

Step 2: Obtain the Source Code

Extract SmartHospital.zip to any folder on the machine. Open Visual Studio 2022 and choose Open Project or Solution, then select SmartHospital.slnx. Alternatively, from the command line:

```
cd SmartHospital
code .
```

Step 3: Restore NuGet Packages

Visual Studio automatically restores on first open. From the command line run dotnet restore in each project folder (SmartHospital.API and SmartHospital.MVC).

Step 4: Configure the Connection String

Open SmartHospital.API/appsettings.json and verify that ConnectionStrings:DefaultConnection points to a reachable SQL Server Express instance. The default value uses DESKTOP-CT4MMTD\SQLEXPRESS01; change the server name to match your host.

Step 5: Create the Database

From the SmartHospital.API folder run:

```
dotnet ef database update
```

This applies the InitialCreate migration and creates the SmartHospitalDB database with the six application tables plus the EF migration history table. The default administrator admin@hospital.com with password Admin@123 is seeded automatically.

Step 6: Run the Web API

From the SmartHospital.API folder run:

```
dotnet run
```

The API listens on https://localhost:7000 and the Swagger UI is served at https://localhost:7000/swagger. Use the Authorize button at the top right to paste a JWT after logging in through /api/Auth/login.

Step 7: Run the MVC Client

In a second terminal, from the SmartHospital.MVC folder run:

```
dotnet run
```

The web front end is served at https://localhost:7001. Ensure the API is already running so that HttpClient calls succeed.

Step 8: First-Time Login (Administrator)

- Open https://localhost:7001 in the browser.
- You are redirected to /Account/Login (Figure 4.2).
- Enter the demo administrator credentials: admin@hospital.com / Admin@123.
- You are redirected to the /Admin dashboard (Figure 4.3).
- From the sidebar you can navigate to Departments, Doctors, Patients, Appointments, Prescriptions, or Bills.

Step 9: Create a Department and a Doctor (Administrator Role)

- Click Departments in the sidebar, then Add Department. Fill in Department Name (for example, Cardiology) and optional Description, then Save.
- Click Doctors, then Add Doctor (Figure 4.4). Select the department, enter Specialization, Experience Years, and Availability, and Save.

Step 10: Patient Self-Registration and Appointment Booking

- Log out through the Logout link at the bottom of the sidebar.
- Click Register here on the login page to navigate to /Account/Register (Figure 4.1).
- Enter Full Name, Email, Password, and Confirm Password, then click Register.
- After successful registration you are redirected to the /Patient dashboard (Figure 4.9) with a green success banner.
- Click Book New Appointment, choose a doctor and date/time slot, then Submit.

Step 11: Doctor Creates and Approves a Prescription

- Log out and log back in as a Doctor account.
- From the /Doctor dashboard (Figure 4.10), click Manage on today's appointment.
- Fill in the Create Prescription form with Diagnosis, Medicines, and optional Notes, then Submit.
- Click Approve on the draft prescription to open the Approve Prescription and Generate Bill page (Figure 4.11).
- Enter the Medicine Charges in rupees and click Approve & Generate Bill. The system persists the approval and inserts a corresponding row in the Bills table.

Step 12: Patient Views Receipt and Marks Bill Paid

- Log in as the Patient account.
- From the /Patient dashboard, click View My Bills to see the list.
- Click on a bill row to open the official receipt (Figure 4.12) which can be printed.
- After offline payment, the administrator can click Mark Paid on the Bills page (Figure 4.8) to flip the status from Unpaid to Paid.

Step 13: Verify Data with SQL Server Management Studio

Open SQL Server Management Studio, connect to DESKTOP-CT4MMTD\SQLEXPRESS01, expand Databases > SmartHospitalDB > Tables, and run queries such as SELECT TOP 1000 * FROM Users (Figure 4.13) to confirm that the BCrypt-hashed password is stored and the Role column is correctly populated.

Step 14: Troubleshooting

- HTTPS certificate warning: run dotnet dev-certs https --trust once to trust the development certificate.

- Port already in use: modify Properties/launchSettings.json for the affected project.
- dotnet-ef not recognised: install it globally with dotnet tool install --global dotnet-ef.
- Build errors after restore: run dotnet clean followed by dotnet build.
- Cannot connect to SQL Server: open SQL Server Configuration Manager and ensure TCP/IP and Named Pipes are enabled for the SQLEXPRESS01 instance, then restart the service.

Achievements

- Successful end-to-end delivery of a role-based Hospital Management System during the Ocean Industrial Training Program (January 6 to June 5, 2026).
- Clean two-project solution structure (Web API plus MVC client) with explicit separation of presentation from business logic.
- Normalised six-entity SQL Server schema with Code First migrations and realistic seed data.
- Three fully functional role-specific dashboards (Admin, Doctor, Patient) with tailored navigation and content.
- Atomic Approve Prescription workflow that simultaneously persists clinical approval and generates the consolidated consultation-plus-medicine bill.
- Printable official bill receipt for patient-facing transparency.
- Direct database validation through Microsoft SQL Server Management Studio confirming BCrypt password hashing and correct role assignments.